



RV College of
Engineering®

Go, change the world®



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Machine Learning Operations (MLOps)

Online Chat Moderation System

Tutorial Report

submitted by

Kavya Jain 1RV23AI045

Arindam Gupta 1RV23AI021

N Mohammed Akhil 1RV23AI061

Ashish R Biradar 1RV23AI022

Artificial Intelligence and Machine Learning

2025-2026



RV College of
Engineering®

Go, change the world®



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project work titled '**Online Chat Moderation System**' is carried out by **Kavya Jain (1RV23AI045), Arindam Gupta (1RV23AI021), N Mohammed Akhil (1RV23AI061), and Ashish R Biradar (1RV24AI020)**, who are Bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of degree of Bachelor of Engineering in Artificial Intelligence and Machine Learning of the Visvesvaraya Technological University, Belagavi during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution for the said degree.

Signature of Faculty
Dr. Anupama Kumar
(Dept. of AIML)

Signature of Faculty
Prof. Manasa M
(Dept. of AIML)

Signature of Head of Department
Dr. B. Satish Babu Professor (Dept.
of AIML)



RV College of
Engineering®

Go, change the world®



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

DECLARATION

We, **Kavya Jain, Arindam Gupta, N Mohammed Akhil, and Ashish R Biradar**, students of fourth semester B.E., Department of AI & ML, RV College of Engineering, Bengaluru, hereby declare that the MLOps Tutorial titled '**Online Chat Moderation System**' has been carried out by us and submitted in partial fulfilment for the award of degree of Bachelor of Engineering in Artificial Intelligence and Machine Learning during the academic year 2025-26. We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date: 29th December, 2025

Name

Signature

Kavya Jain

Arindam Gupta

N Mohammed Akhil

Ashish R Biradar



ABSTRACT

The rapid growth of online platforms has led to a significant increase in user-generated content, which includes harmful, abusive, and toxic comments. Traditional content moderation techniques rely heavily on static rule-based filters or manually blacklisting them, which are ineffective against evolving language patterns, slang, and contextual toxicity. This creates serious concerns related to user safety, mental well-being, and ethical governance of digital platforms.

The objective of this work is to design and implement an intelligent, scalable, and continuously learning chat moderation system capable of detecting toxic content in real time while maintaining transparency and ethical accountability. The proposed system integrates advanced natural language processing models with modern MLOps practices to ensure reliable deployment, continuous monitoring, and automated updates.

The methodology begins with large-scale data collection, followed by Exploratory Data Analysis, Data Cleaning, and Labeling. Data Versioning is handled using DVC to ensure traceability. Multiple machine learning and deep learning models, including Logistic Regression, LSTM, and BERT, are trained and evaluated. MLflow is used for experiment tracking, metric logging, and model version management. Explainability is achieved using SHAP to understand and justify model predictions. The best-performing model is deployed through a FAST API-based service and monitored in real time using Evidently AI for data and prediction drift. Continuous integration and deployment are achieved using GitHub Actions and Docker, with cloud hosting on Render.

Experimental results show that transformer-based models, particularly BERT, outperform traditional approaches in terms of precision, recall, and robustness to evolving language patterns. Drift monitoring further ensures sustained performance over time.

Future work includes multilingual moderation, multimodal toxicity detection, and integration of reinforcement learning for adaptive moderation policies. This system demonstrates a practical and ethical approach to deploying AI-driven content moderation in real-world environments.



TABLE OF CONTENT

Chapter No.	Title	Page No.
	Certificate	i
	Declaration	ii
	Abstract	iii
	Table of Contents	iv
	Table of Figures	vi
	Table of Tables	vi
	Glossary	vii
1	Introduction	1
2	Problem Statement	4
3	Motivation	4
4	Objectives	5
5	Literature Review	6
6	Methodology	8
7	Architecture	11
8	Tools and Techniques	17
9	Implementation	21
10	Prototype	26
11	Results and Discussion	30
12	Challenges Faced	36
13	Conclusion	38
14	References	39



TABLE OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Flowchart of the System	13
Figure 2	Layered Architecture of MLOps-Based Chat Moderation System	18
Figure 3	MLflow-Based Experiment Tracking and Model Selection Workflow	32
Figure 4	SHAP-Based Feature Contribution Analysis	34
Figure 5	FastAPI-Based Model Inference Architecture	36
Figure 6	CI/CD Pipeline using GitHub Actions	37
Figure 7	API Overview	39
Figure 8	Prediction Execution and Response	40
Figure 9	Data Schemas and Validation	41
Figure 10	Model Performance Comparison	45
Figure 11	MLflow Experiment Dashboard	47
Figure 12	DistilBERT MLflow Metrics	48
Figure 13	SHAP Analysis	49
Figure 14	Confusion Matrix for DistilBERT	50
Figure 15	ROC Curve	51
Figure 16	Precision–Recall Curve	51

TABLE OF TABLES

Table No.	Table Title	Page No.
Table 1	Difference Between Existing and Proposed System	3
Table 2	Architecture Layers and MLOps Tools	22
Table 3	Tools and Techniques	28
Table 4	Machine Learning Models Used	31
Table 5	Deployment Tools and Responsibilities	37



GLOSSARY

Term	Description	Term
AI	Artificial Intelligence systems that simulate human intelligence	AI
ML	Machine Learning algorithms that learn patterns from data	ML
MLOps	Practices that manage the ML lifecycle from development to deployment	MLOps
Toxic Content	Harmful language including hate speech, harassment, or abuse	Toxic Content
NLP	Natural Language Processing AI techniques for text understanding	NLP
BERT	Transformer-based language model for contextual text understanding	BERT
DistilBERT	Lightweight version of BERT optimized for efficiency	DistilBERT
SHAP	Explainable AI method to interpret model predictions	SHAP
DVC	Data Version Control tool for tracking dataset versions	DVC
MLflow	MLOps tool for experiment tracking and model registry	MLflow
FastAPI	Python framework for building high-performance APIs	FastAPI

INTRODUCTION

Background and Overview

Online communication platforms such as social media, forums, and messaging applications have become an essential part of daily life. People use these platforms to share opinions, connect with others, and exchange information instantly. However, along with this rapid growth, there has been a significant rise in toxic content such as hate speech, abusive language, harassment, and offensive comments.

Traditionally, such content has been handled by human moderators. While manual moderation can be effective, it suffers from major limitations. It is time-consuming, expensive to scale, and emotionally exhausting for moderators who are repeatedly exposed to harmful content. As platforms grow, manual moderation alone becomes impractical.

Artificial Intelligence (AI) based moderation systems offer a scalable solution by automatically analyzing text and identifying toxic behavior. However, deploying these AI systems in real-world environments introduces new challenges. Models may become biased, their decisions may lack transparency, and their performance can degrade over time due to changing language patterns, slang, and cultural context. Hence, a robust and continuously managed AI system is required.

Existing System

Most existing content moderation systems follow a traditional and limited approach. These systems usually rely on:

- **Keyword-based filtering**, where predefined offensive words are blocked
- **Static machine learning models**, trained once and rarely updated
- **Manual rule updates**, requiring human intervention for improvements

While these methods are simple to implement, they have serious drawbacks. Keyword-based systems cannot understand context, meaning they may block harmless messages or fail to detect cleverly worded abuse. Static models do not adapt to new slang or evolving toxic behavior.



Furthermore, these systems do not provide explanations for why a particular message was flagged, reducing trust and transparency.

Proposed System

The proposed system introduces an **end-to-end AI driven chat moderation pipeline integrated with MLOps practices**. Instead of treating moderation as a one-time model deployment, the system treats it as a continuously evolving service.

The system uses advanced models, such as transformer-based architectures, to understand context and semantics. MLOps tools are used to ensure continuous model improvement, automated deployment, real-time monitoring, and ethical governance. The system can detect data drift, explain predictions, and automatically deploy updated models without manual intervention.

Aspect	Existing System	Proposed System
Adaptability	Static models	Continuous learning and retraining
Explainability	Not available	SHAP-based explanations
Deployment	Manual updates	Automated CI/CD pipelines
Monitoring	No monitoring	Drift detection and performance tracking
Governance	Limited control	Transparent and ethical AI framework

Table 1 - Difference Between Existing and Proposed System

This comparison clearly shows that the proposed system is more robust, adaptive, and suitable for real-world deployment.

Novelty

The novelty of this work lies in the **tight integration of modern models with a complete MLOps lifecycle**. Unlike conventional moderation systems that focus only on model accuracy, this approach emphasizes sustainability, transparency, and reliability.



Key innovative aspects include experiment tracking, data and model versioning, explainable AI techniques, automated deployment pipelines, and drift monitoring. By combining these elements, the system evolves with changing language patterns and maintains consistent performance over time. This shifts moderation from a static classifier to a continuously managed AI service.

Societal Impact

The proposed system has a strong positive societal impact. By reducing the spread of harmful and abusive content, it helps create safer and more inclusive online communities. Automated moderation reduces human exposure to toxic material, protecting moderators' mental well-being.

Additionally, the use of explainable and ethical AI builds user trust and accountability. Transparent decision-making ensures fairness and supports responsible AI deployment aligned with legal and ethical standards.



PROBLEM STATMENT

The rapid expansion of online communication platforms has led to a significant increase in toxic content, including hate speech, harassment, and abusive language. Existing moderation mechanisms are largely based on static machine learning models and rule-based filtering techniques that are trained once and rarely updated. These approaches struggle to cope with continuously evolving language patterns, emerging slang, and context-dependent expressions of harmful intent. In addition, the decision-making processes of many AI-based moderation systems lack transparency, making it difficult to understand or justify classification outcomes. This lack of adaptability and explainability raises concerns related to reliability, fairness, and accountability in real-world moderation scenarios. As a result, maintaining consistent and trustworthy content moderation in dynamic online environments remains a major challenge.

MOTIVATION

The motivation for this work arises from the growing limitations of traditional and static machine learning-based moderation systems in handling modern online toxicity. Language on digital platforms evolves rapidly, with harmful intent often expressed through sarcasm, abbreviations, emojis, and coded expressions that cannot be effectively captured by keyword-based filters or outdated models. Even advanced ML models experience performance degradation over time due to data and concept drift, leading to incorrect predictions and reduced trust in automated systems. Furthermore, the lack of explainability in many AI models raises ethical concerns, especially in applications related to free speech and user safety. These challenges highlight the importance of adopting modern ML tools and MLOps practices such as experiment tracking, model versioning, explainable AI, drift detection, and automated CI/CD pipelines. The motivation of this project is to build a reliable, transparent, and adaptive moderation system that remains effective throughout its lifecycle in real-world deployment.



OBJECTIVES

Objective 1: To build an accurate toxic comment detection model

The first objective is to develop a high-performing machine learning model capable of detecting toxic and non-toxic comments in real time. Advanced techniques, including word embeddings and transformer-based representations, are used to capture contextual meaning and intent. ML training frameworks and evaluation tools are employed to optimize accuracy, precision, and recall.

Objective 2: To compare traditional machine learning and deep learning models

This objective aims to analyze and compare multiple ML approaches such as Logistic Regression, Naive Bayes, LSTM, and transformer-based models i.e. BERT. Using ML experimentation tools, each model is evaluated on performance metrics, training time, and interpretability. This comparison helps identify the most suitable model for deployment under real-world constraints.

Objective 3: To implement an end-to-end MLOps automation pipeline

A major objective is to integrate ML tools into a complete MLOps pipeline that supports dataset versioning, experiment tracking, automated model training, and CI/CD-based deployment. Tools such as MLflow, Docker, and workflow schedulers enable reproducibility, scalability, and faster iteration while minimizing manual intervention.

Objective 4: To ensure explainable and ethical AI decisions

This objective focuses on incorporating explainable AI tools such as SHAP to interpret model predictions. These tools provide feature-level explanations that justify why a particular message was classified as toxic or non-toxic. Explainability enhances trust, supports ethical AI usage, and enables compliance with responsible AI guidelines.

Objective 5: To continuously monitor data and prediction drift

The final objective is to use ML monitoring tools to track data drift and model performance degradation over time. Drift detection tools identify changes in language usage and user behavior, triggering automated retraining and redeployment of models. This ensures long-term reliability and consistent moderation quality in dynamic online environments.



LITERATURE REVIEW

1) Z. Zhao et al., 2022

This work focuses on a real issue in toxic comment detection called **identity term bias**, where models incorrectly learn that certain identity-related words are toxic even when the sentence is not harmful. The authors propose a technique that uses **subjectivity signals** to reduce this bias and show that their improved BERT-based approach performs better and is fairer. This paper is important for your project because it supports the **governance goal**: moderation should be accurate but also **non-discriminatory**, and bias reduction should be part of evaluation. [ScienceDirect](#)

2) Y. Cao et al., 2024

This paper explains that toxicity detection alone does not fully solve the real needs of moderation teams. The authors analyze the gap between “high accuracy toxic classifiers” and the actual workflows of moderators, showing that real moderation requires more context, better support tools, and user-centric thinking. This directly supports your report argument that a moderation system must include **explainability, monitoring, and governance**, not only classification accuracy. [ACL Anthology](#)

3) Z. Hu et al., 2024

This work introduces a different viewpoint: instead of always running a separate toxicity classifier, the authors show how toxicity can be detected using **signals from an existing aligned LLM** (“toxicity detection for free”). The paper highlights practical challenges in real-world toxicity detection such as **rare toxic examples** and the need for strong performance at **low false positive rates**. This is useful in your discussion because it supports why production moderation needs careful thresholding, reliability, and monitoring beyond basic metrics. [arXiv+1](#)

4) J. Chan et al., 2024

This paper studies the problem of **disguised toxicity**—users intentionally hide abusive meaning through creative spellings, formatting tricks, or masked words. The authors propose a preprocessing module aimed at improving toxicity detection systems. This connects strongly to your project’s drift monitoring idea: as users invent new disguised patterns, models can degrade.



It supports your need for **continuous updates**, drift detection, and robust preprocessing in the methodology. [PMC](#)

5) M. Zangl et al., 2025

This paper provides a broader and modern perspective on deploying toxicity detection in real settings, focusing on **transparent and explainable AI-driven toxicity systems**. It highlights that explainability methods often lack standard evaluation, and that deployment requires structured best practices. This directly supports your system design: using tools like **SHAP for explanations** and having a monitored and auditable pipeline is necessary for trustworthy moderation. [Springer](#)

METHODOLOGY

The methodology describes the complete workflow followed to design, train, deploy, and monitor the AI-based chat moderation system using modern machine learning and MLOps tools. The approach ensures scalability, transparency, and continuous performance improvement.

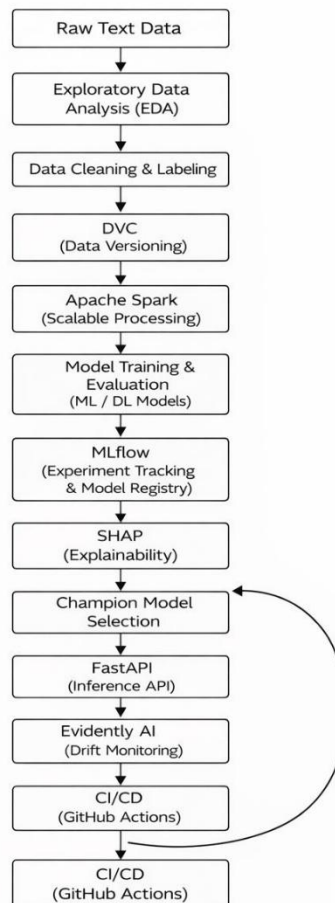


Figure 1 – Flowchart of the system

Figure 1 illustrates the complete end-to-end pipeline of the proposed moderation system. The workflow begins with raw text data and progresses through data preprocessing, scalable processing, model training, evaluation, deployment, and continuous monitoring. Each stage is supported by dedicated ML and MLOps tools to ensure automation, reproducibility, and reliability.



Methodology Explanation

The methodology begins with the **collection of raw textual data**, which consists of user generated comments obtained from online communication platforms such as social media or discussion forums. This data is unstructured and may contain spelling variations, emojis, slang, and informal expressions.

The collected data is first analyzed using **Exploratory Data Analysis (EDA)**. EDA is a statistical and visual analysis process used to understand the structure of the dataset. In this stage, the distribution of toxic and non-toxic classes is examined, frequently occurring words are identified, and noise such as emojis, symbols, and repeated characters is detected. EDA helps in understanding data imbalance and guides preprocessing decisions.

Following this, **data cleaning and labeling** are performed. Data cleaning involves removing unnecessary symbols, URLs, and extra spaces, converting text to a standard format, and handling missing or inconsistent values. Labeling assigns meaningful tags such as *toxic* or *non-toxic* to each comment, which is essential for supervised machine learning. To ensure reproducibility and consistency across experiments, **Data Version Control (DVC)** is used. DVC is a tool that tracks dataset versions similarly to how Git tracks code, allowing researchers to reproduce experiments even when data changes over time.

To efficiently process large volumes of text data, **Apache Spark** is employed. Spark is a distributed data processing framework designed for handling big data workloads. It enables parallel processing of text data, making preprocessing and feature extraction faster and scalable when dealing with large datasets.

Once the data is prepared, **multiple machine learning and deep learning models** are trained using the processed dataset. These models may include traditional classifiers and neural network-based architectures. During this phase, **MLflow** is used for experiment tracking. MLflow is an MLOps tool that records model parameters, training configurations, performance metrics, and output artifacts. This systematic logging allows easy comparison of different models and helps in selecting the most effective one.



To improve transparency and trust in model predictions, **SHAP (SHapley Additive Explanations)** is integrated. SHAP is an explainable AI technique that explains individual predictions by showing how each word or feature contributes positively or negatively to the final decision. This helps in understanding why a comment is classified as toxic or non-toxic, addressing ethical and accountability concerns.

Based on evaluation metrics such as accuracy, precision, recall, and F1-score, the best-performing model is selected as the **Champion Model**. The Champion Model represents the most reliable model ready for deployment among all trained candidates.

The selected model is deployed using **FastAPI**, a lightweight web framework for building high-performance RESTful APIs. FastAPI enables real-time inference by allowing external applications to send text inputs and receive toxicity predictions instantly.

After deployment, **Evidently AI** is used for continuous monitoring. Evidently AI is a model monitoring tool that detects data drift and changes in prediction behavior over time. It helps identify when the model's performance may degrade due to evolving language patterns or user behavior.

Finally, **CI/CD (Continuous Integration and Continuous Deployment) pipelines** automate the retraining, testing, and redeployment process. CI/CD ensures that whenever significant data drift or performance degradation is detected, updated models can be tested and deployed automatically without manual intervention, maintaining long-term system reliability.

ARCHITECTURE

The proposed chat moderation system follows a **layered MLOps-driven architecture** that separates data handling, model development, serving, monitoring, and deployment concerns. This layered design improves scalability, traceability, reproducibility, and long-term maintainability of the system.

Architecture Flow

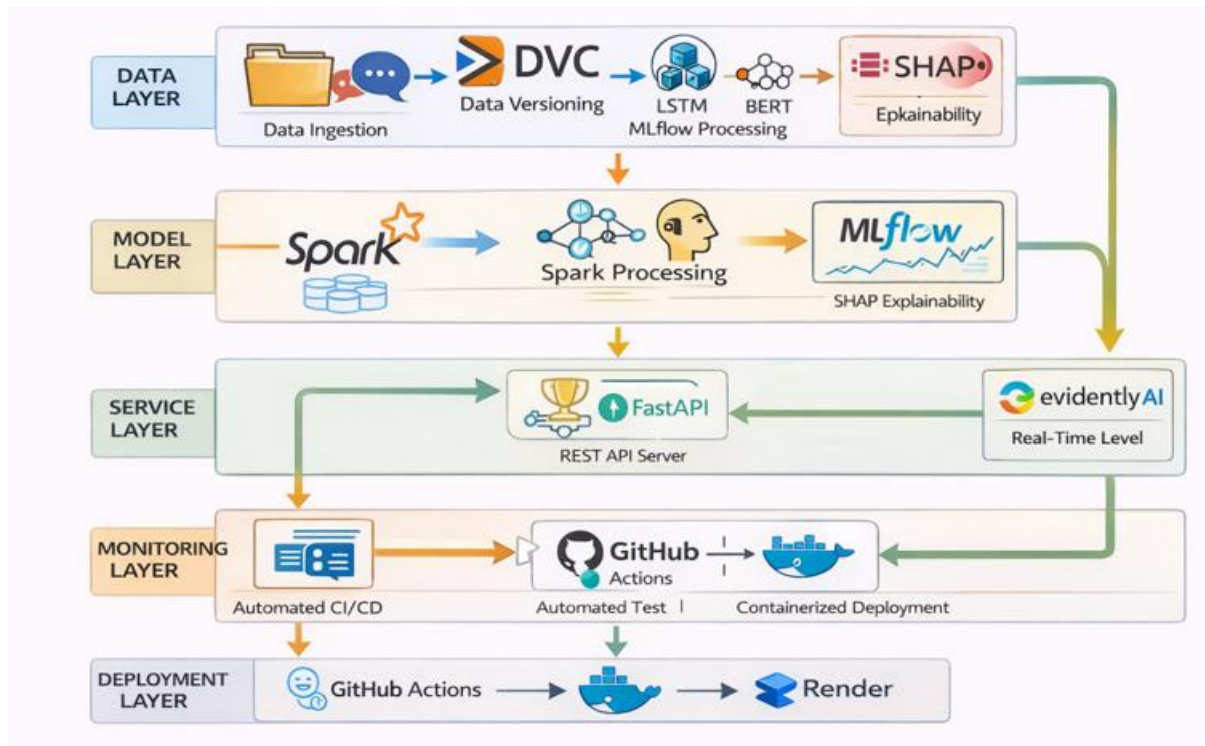


Figure 2 – Layered Architecture of MLOps – Based Chat Moderation System

The diagram illustrates a sequential yet interconnected flow starting from data ingestion and ending with deployment and monitoring. Each layer performs a specific responsibility and communicates with adjacent layers through well-defined interfaces. Feedback loops from monitoring to retraining ensure continuous system improvement.

Data Layer

Components:

- Data Ingestion



- Data Version Control (DVC)
- Apache Spark

Functionality and Tool Usage

The **Data Layer** is responsible for collecting, preprocessing, versioning, and preparing textual data for model training. Raw user comments are ingested from online platforms and stored as unstructured text data.

Before training, datasets are tracked using **DVC (Data Version Control)**. DVC ensures that every dataset version used for training is linked to a specific experiment and model version. This is critical in MLOps because changes in data can directly impact model performance. Using DVC, previous datasets can be reproduced and compared reliably.

For scalable preprocessing, **Apache Spark** is employed. Spark distributes data processing across multiple nodes, enabling efficient handling of large volumes of text. Operations such as tokenization, filtering, and feature extraction are parallelized, significantly reducing preprocessing time compared to single-machine processing.

In **Figure 2**, the Data Layer appears at the beginning of the pipeline. Arrows from the Data Layer to the Model Layer represent the transfer of cleaned, versioned, and processed datasets for training.

Model Layer

Components:

- Machine Learning and Deep Learning Models
- MLflow
- SHAP

Functionality and Tool Usage

The **Model Layer** focuses on training, evaluating, comparing, and explaining machine learning models. Multiple models such as traditional classifiers, LSTM, and transformer-based models are trained using the processed dataset.



MLflow plays a central role in this layer by tracking experiments. For every training run, MLflow logs:

- Model parameters
- Training configurations
- Performance metrics (accuracy, precision, recall, F1-score)
- Model artifacts

This systematic tracking allows researchers to compare models objectively and identify the most reliable one. MLflow also maintains a model registry, which helps in promoting the best-performing model to production status.

To address transparency and ethical concerns, **SHAP (SHapley Additive Explanations)** is integrated. SHAP explains individual predictions by quantifying how each word contributes to a model's output. This makes the moderation decisions interpretable and auditable, which is crucial for responsible AI systems.

In **Figure 2**, the Model Layer is shown as a centralized block connected to both the Data Layer and Service Layer. The internal arrows indicate model comparison and selection, while SHAP outputs feed into monitoring and governance processes.

Service Layer

Components:

- FastAPI Inference API

Functionality and Tool Usage

The **Service Layer** exposes the trained model as a real-time inference service. The selected **Champion Model** is deployed using **FastAPI**, a high-performance Python web framework.

FastAPI provides RESTful endpoints that accept user text as input and return toxicity predictions as output. It supports low-latency inference, making it suitable for real-time chat moderation



scenarios. This layer acts as the interface between the AI system and external applications such as chat platforms or dashboards.

In **Figure 2**, the Service Layer appears between the Model Layer and Deployment Layer. Bidirectional arrows indicate that updated models can be redeployed seamlessly without changing the API interface.

Monitoring Layer

Components:

- Evidently AI

Functionality and Tool Usage

The **Monitoring Layer** ensures continuous observation of model behavior after deployment.

Evidently AI monitors:

- Input data distribution
- Prediction distribution
- Data drift and concept drift

When language usage changes over time, Evidently AI detects deviations from training data patterns. These alerts trigger retraining workflows, preventing silent model degradation.

In **Figure 2**, the Monitoring Layer forms a feedback loop back to the Data and Model Layers. This loop represents continuous learning and adaptive model management, a key principle of MLOps.

Deployment Layer

Components:

- GitHub Actions
- Docker
- Render



Functionality and Tool Usage

The **Deployment Layer** automates testing, packaging, and deployment using CI/CD practices.

- **GitHub Actions** automates code integration, testing, and pipeline execution whenever changes are committed.
- **Docker** containerizes the application, ensuring consistent runtime environments across development and production.
- **Render** hosts the containerized FastAPI service in the cloud, enabling scalable and reliable access.

Together, these tools ensure rapid, error-free deployment with minimal manual intervention.

In **Figure 2**, the Deployment Layer appears at the bottom, receiving artifacts from the Service Layer and feeding deployment status back into monitoring systems.

Layer	Tools Used	Purpose
Data Layer	DVC, Spark	Versioned, scalable data processing
Model Layer	MLflow, SHAP	Experiment tracking and explainability
Service Layer	FastAPI	Real-time inference
Monitoring Layer	Evidently AI	Drift detection and monitoring

Table 2 - Architecture Layers and MLOps Tools

Architectural Advantages

Scalability

The architecture supports scalability by using **distributed data processing with Apache Spark** and **containerized deployment using Docker**. This allows the system to handle increasing volumes of data and user requests efficiently without degrading performance.

Traceability

It is achieved through **dataset versioning with DVC** and **experiment tracking with MLflow**. Every model can be traced back to the exact data version, parameters, and metrics used during training, ensuring reproducibility and auditability.



Maintainability

The system follows a **modular layered design**, where each layer (data, model, service, monitoring, deployment) has a clearly defined responsibility. This separation makes it easier to update, debug, or replace individual components without affecting the entire system.

Ethical AI

Ethical AI is supported through **explainable predictions using SHAP** and **continuous monitoring with Evidently AI**. These tools ensure transparency in decision-making and help detect bias, drift, or unexpected behavior in deployed models over time.



TOOLS AND TECHNIQUES

This project employs a comprehensive set of machine learning, data engineering, and MLOps tools to design, deploy, and maintain a robust chat moderation system. The selected tools collectively support the entire machine learning lifecycle, including data management, model development, experiment tracking, explainability, deployment, and continuous monitoring. The primary emphasis of this work is on machine learning tools that enable reliable, explainable, and continuously improving AI systems.

Scikit-learn

Scikit-learn is a widely used machine learning library in Python that provides efficient implementations of classical machine learning algorithms. In this project, scikit-learn is used to develop baseline models such as Logistic Regression. These models serve as an initial benchmark for evaluating performance against more advanced deep learning approaches. Scikit-learn is also used to compute key evaluation metrics such as accuracy, precision, recall, and F1-score. Its simplicity and interpretability make it suitable for understanding baseline behavior before deploying complex models.

TensorFlow

TensorFlow is a deep learning framework used to build and train neural network models. In this project, TensorFlow is used to implement LSTM-based models for toxic comment classification. LSTM networks are capable of capturing sequential dependencies in text, making them effective for understanding sentence structure and contextual flow. TensorFlow also provides flexibility in model training, optimization, and GPU acceleration, which is essential for handling large datasets and complex neural architectures.

Transformers (BERT)

Transformer-based models, specifically BERT (Bidirectional Encoder Representations from Transformers), form the core intelligence of the proposed system. BERT is capable of understanding contextual meaning by processing text bidirectionally. This allows the model to accurately identify subtle toxic patterns, sarcasm, and context-dependent language. In this project,



BERT consistently outperformed traditional machine learning and LSTM models, making it the champion model selected for deployment. Its strong generalization ability makes it suitable for real-world moderation tasks.

MLflow

MLflow is a central MLOps tool used for experiment tracking, model versioning, and lifecycle management. In this project, MLflow logs all model training experiments, including parameters, metrics, artifacts, and dataset versions. It enables easy comparison between different models and training runs. MLflow's model registry is used to manage multiple versions of trained models and to promote the best-performing model to production. This ensures reproducibility, traceability, and controlled deployment of models.

DVC (Data Version Control)

DVC is used to manage and version large datasets that cannot be efficiently handled by Git alone. In this project, DVC tracks raw, cleaned, and processed datasets, ensuring that every model can be traced back to the exact data version used during training. This plays a critical role in reproducibility and auditability. DVC integrates seamlessly with Git, allowing dataset changes to be linked to code changes, which is essential for structured MLOps workflows.

Apache Spark

Apache Spark is a distributed data processing framework used to handle large-scale text datasets efficiently. In this project, Spark is employed for scalable data preprocessing tasks such as cleaning, transformation, and aggregation. Spark enables parallel processing, significantly reducing computation time when working with large volumes of user-generated content. This ensures that data preparation remains efficient even as dataset size increases.

SHAP (SHapley Additive exPlanations)

SHAP is an explainable AI tool used to interpret model predictions. In the context of chat moderation, explainability is essential for ethical AI deployment. SHAP identifies which words or tokens contribute most to a toxicity prediction, providing transparency into model behavior. In this project, SHAP is applied to the trained models to validate that predictions are based on meaningful



linguistic features rather than biased or irrelevant patterns. This supports governance, fairness, and trust in the system.

FastAPI

FastAPI is a high-performance web framework used to serve the trained machine learning model as a REST API. In this project, FastAPI acts as the inference layer, receiving user text as input and returning toxicity predictions in real time. It enables low-latency responses, automatic

API documentation, and seamless integration with ML models. FastAPI ensures that the moderation system can be accessed by external applications in a scalable and efficient manner.

Evidently AI

Evidently AI is used for monitoring data drift and prediction drift in production. Over time, language usage and toxicity patterns evolve, which can degrade model performance. Evidently AI continuously compares live incoming data with training data and generates drift reports. When significant drift is detected, it signals the need for retraining. This ensures that the deployed model remains accurate and reliable in dynamic real-world environments.

GitHub Actions

GitHub Actions is used to automate the CI/CD pipeline. Whenever code or model updates are pushed to the repository, GitHub Actions automatically triggers workflows for testing, building, and deployment. In this project, GitHub Actions ensures that model updates and API changes are deployed consistently without manual intervention. This reduces deployment errors and improves system reliability.

Docker

Docker is a containerization platform used to package the FastAPI application, trained model, and all dependencies into a single container. This ensures consistency across development, testing, and production environments. Docker simplifies deployment and allows the system to scale efficiently across different platforms without configuration issues.



Render

Render is a cloud hosting platform used to deploy the containerized FastAPI application. It provides automatic builds, environment management, and scalability. In this project, Render hosts the live moderation API, making it accessible for real-time usage while supporting continuous deployment from GitHub.

Category	Tools Used	Purpose
ML Modeling	Scikit-learn, TensorFlow, BERT	Toxicity classification
Experiment Tracking	MLflow	Model comparison and versioning
Data Management	DVC, Spark	Data versioning and processing
Explainability	SHAP	Ethical AI and transparency
Serving	FastAPI	Real-time inference
Monitoring	Evidently AI	Drift detection
CI/CD	GitHub Actions	Automated deployment
Deployment	Docker, Render	Scalable hosting

Table 3 – Tools and Techniques



IMPLEMENTATION

The implementation of the chat moderation system followed a structured MLOps-driven workflow, ensuring reproducibility, automation, and scalability across the model lifecycle.

Development Environment

The system was implemented using **Python**, which provided extensive support for machine learning, natural language processing, and web-based services. The implementation followed a modular design, allowing independent development of data processing, model training, serving, and deployment components.

Model Development and Training

Machine learning and deep learning models were developed to classify text comments as toxic or non-toxic. Traditional models were implemented using **Scikit-learn** to establish baseline performance and ensure interpretability. Advanced deep learning models were developed using **TensorFlow** to capture contextual and sequential dependencies in text.

The models were trained on preprocessed and versioned datasets, and evaluated using standard classification metrics such as accuracy, precision, recall, and F1-score.

Model Type	Framework	Purpose
Logistic Regression	Scikit-learn	Baseline model
Random Forest	Scikit-learn	Decision stumps
LSTM	TensorFlow	Sequential text understanding
Transformer-based Model	TensorFlow	Context-aware classification

Table 4 - Machine Learning Models Used

Table 4 summarizes the models used during training and highlights the progression from traditional machine learning to deep learning approaches.

Experiment Tracking with MLflow

All model training experiments were tracked using **MLflow**. MLflow was used to log hyperparameters, evaluation metrics, and trained model artifacts for each experiment. This enabled systematic comparison across multiple training runs and ensured reproducibility of results.

MLflow also supported model versioning, which facilitated the identification and promotion of the best-performing model, referred to as the **Champion Model**.

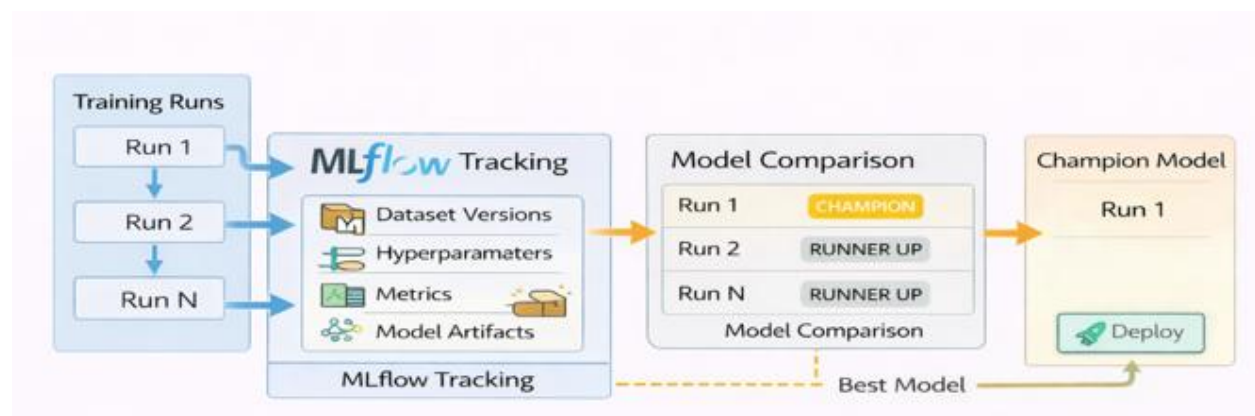


Figure 3 - MLflow-Based Experiment Tracking and Model Selection Workflow

Figure 3 illustrates how multiple model experiments were logged and compared using MLflow, leading to the selection of the Champion Model for deployment.

Explainability using SHAP

To support transparency and ethical AI practices, **SHAP (SHapley Additive Explanations)** was integrated into the evaluation pipeline. SHAP was used to analyze individual predictions by identifying how specific words contributed to the final classification.

This explainability mechanism enabled interpretation of model behavior and improved trust in automated moderation decisions.

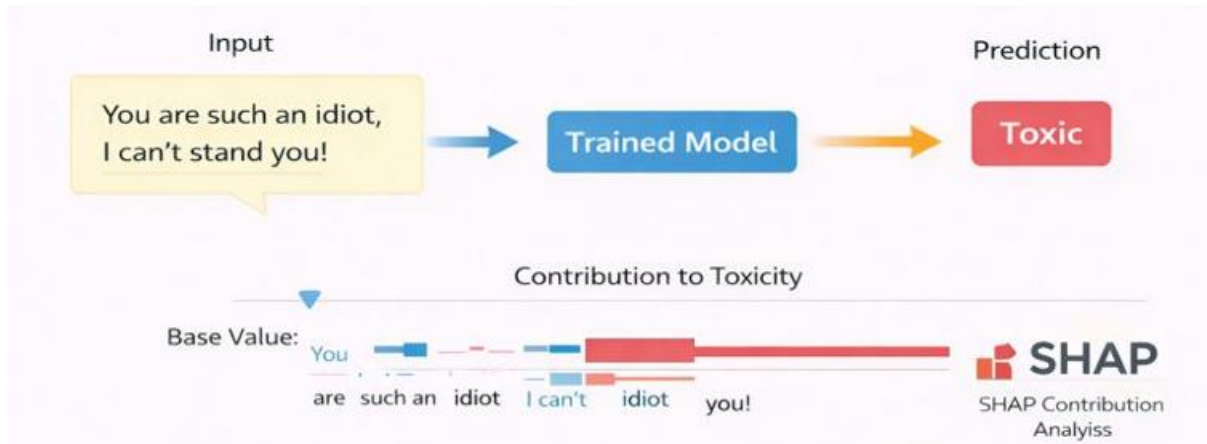


Figure 4 - SHAP-Based Feature Contribution Analysis

Figure 4 demonstrates how SHAP values were used to explain the influence of individual textual features on model predictions.

Model Serving using FastAPI

The selected Champion Model was deployed as a real-time inference service using **FastAPI**. FastAPI was used to expose RESTful endpoints that accept text input and return toxicity predictions.

This service-oriented implementation enabled seamless integration with external applications such as chat platforms and dashboards.

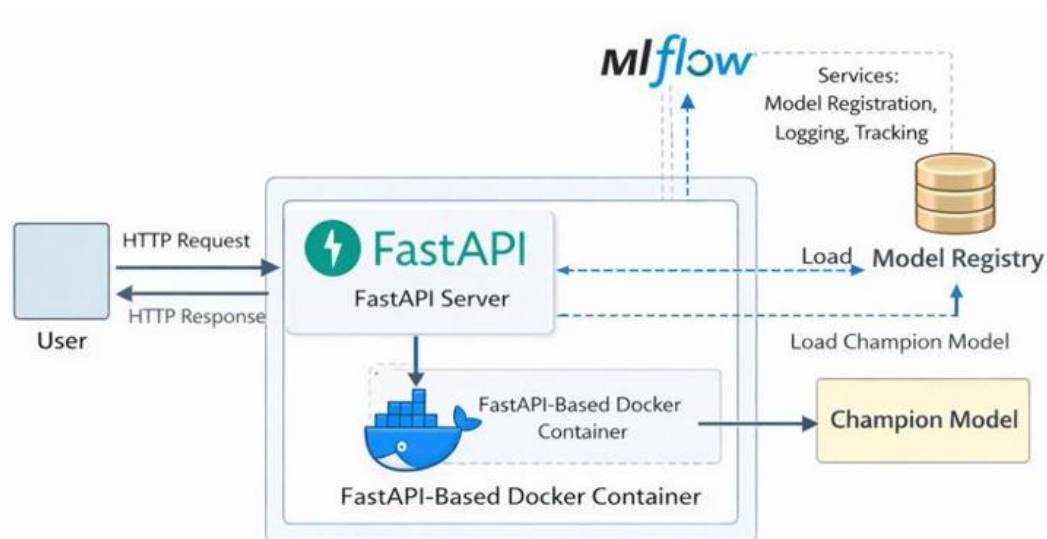


Figure 5 - FastAPI-Based Model Inference Architecture

Figure 5 illustrates the interaction between client applications and the FastAPI inference service hosting the trained model.

Containerization using Docker

To ensure consistency across development and production environments, the application was containerized using **Docker**. Containerization encapsulated all dependencies, configurations, and the inference service into a single portable unit.

This approach simplified deployment and improved system reliability.

CI/CD Automation using GitHub Actions

Continuous integration and continuous deployment were automated using **GitHub Actions**. The CI/CD pipeline was configured to perform automated testing, container image creation, and deployment upon code updates.

This automation reduced manual intervention and ensured faster and more reliable releases.

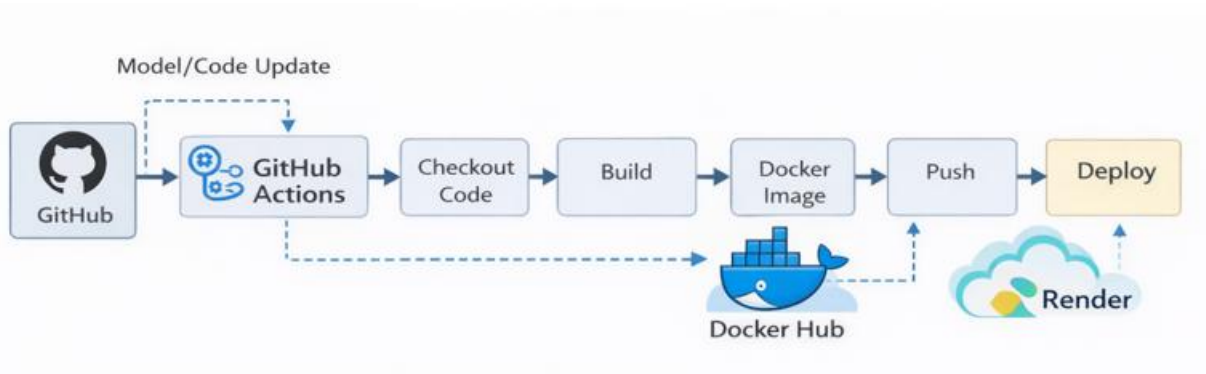


Figure 6 - CI/CD Pipeline using GitHub Actions

Figure 6 shows the automated workflow from code commit to deployment, highlighting the role of GitHub Actions in pipeline automation.

Cloud Deployment using Render

The containerized FastAPI application was deployed on the **Render** cloud platform. Render was used to host the moderation service and manage application availability in a production environment.



This deployment enabled scalable and continuous access to the moderation system.

Tool	Role
Docker	Containerization
GitHub Actions	CI/CD automation
Render	Cloud hosting

Table 5 - Deployment Tools and Responsibilities

Table 5 summarizes the deployment-related tools and their respective responsibilities in the production environment.

PROTOTYPE

The prototype demonstrates a **fully deployed, production-ready toxic comment moderation service** implemented using FastAPI and transformer-based machine learning models. It validates the complete MLOps lifecycle by exposing a real-time inference API, enabling structured request–response interaction, and showcasing automated deployment on a cloud platform. The prototype is visualized through three interface screenshots generated by the FastAPI OpenAPI (Swagger UI).



Figure 7 - API Overview

A. Prototype Interface

The Figure 7 presents the **Swagger-based API documentation interface** automatically generated by FastAPI. This interface serves as a live and interactive description of the deployed toxic comment classification service.

At the top of the interface, the service is titled “**Toxic Comment Classifier**”, clearly indicating the purpose of the deployed system. The version information and OpenAPI specification (OAS 3.1) confirm that the service adheres to modern API standards. Additionally, the label “**MLOps Champion Model: DistilBERT Inference Service**” highlights that the deployed model is a distilled transformer model selected through an MLOps-driven champion selection process.

Two primary endpoints are exposed:

- A **GET endpoint** for health checking, which allows automated systems and monitoring tools to verify service availability.
- A **POST endpoint (/predict)** that performs real-time toxicity prediction on user-provided text.

This interface demonstrates how machine learning models are transformed into **consumable AI services**, enabling seamless integration with external applications such as chat platforms or moderation dashboards.

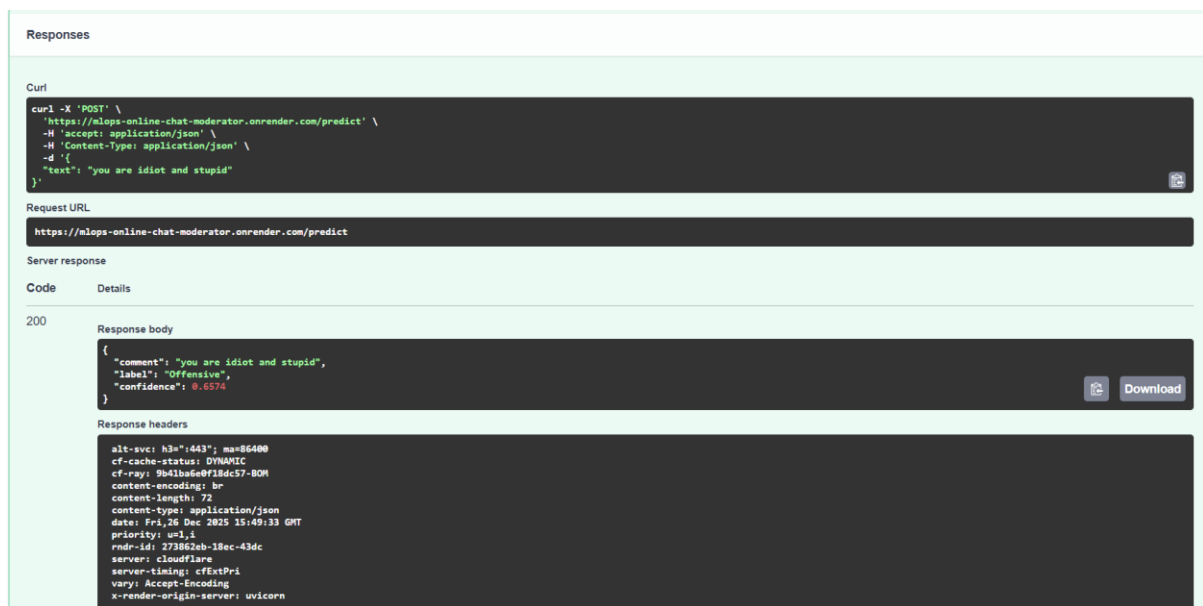


Figure 8 - Prediction Execution and Response

B. Prototype Interface

The Figure 8 illustrates the **core functionality of the prototype**, where a sample toxic comment is submitted to the prediction endpoint using an HTTP POST request.

A structured JSON request containing the input text:

“you are idiot and stupid”

is sent to the /predict endpoint. This request simulates a real-world user comment submitted for moderation. The prototype clearly displays the equivalent curl command, emphasizing that the API can be accessed programmatically by any client application.

Upon execution, the server returns a **200 OK response**, indicating successful processing. The response body contains:

- The original comment
- The predicted label (“**Offensive**”)
- A confidence score (**0.6574**), representing the model’s certainty

This output confirms that the deployed transformer-based model is functioning correctly in production. The presence of confidence scores supports moderation decision-making by allowing threshold-based actions, such as warning users or escalating content for manual review.

Additionally, the response headers reveal that the service is hosted on a cloud infrastructure using **Uvicorn** as the ASGI server and **Cloudflare** for edge handling, demonstrating real-world deployment readiness.



Figure 9 - Data Schemas and Validation



C. Prototype Interface

The Figure 9 presents the **data schema definitions** used by the API, which are automatically generated from FastAPI's Pydantic models. This section is critical for ensuring robustness, validation, and consistency in machine learning inference services.

The **CommentRequest schema** defines the required input structure, enforcing that the request must contain a valid text string. This prevents malformed inputs from reaching the model.

The **PredictionResponse schema** specifies the structured output returned by the model, including the predicted label and confidence score. This ensures consistent responses across all client applications.

Additionally, validation error schemas such as **HTTPValidationError** and **ValidationError** are included. These schemas demonstrate how the system gracefully handles invalid inputs, improving reliability and user experience.

This strict schema enforcement reflects best practices in MLOps, where model serving is treated as a production-grade software component rather than an experimental script.

The prototype represents a **complete end-to-end MLOps-enabled machine learning system in production**. A transformer-based toxicity detection model, selected as the champion model through MLflow experiment tracking, is deployed as a RESTful service using FastAPI. The service is containerized using Docker and hosted on a cloud platform, making it accessible through standard HTTP requests.

RESULTS AND DISCUSSION

This section presents a detailed analysis of the experimental results obtained from multiple machine learning models developed for toxic comment classification. The evaluation focuses on comparative performance, explainability, and production readiness, with particular emphasis on MLOps-driven decision-making.

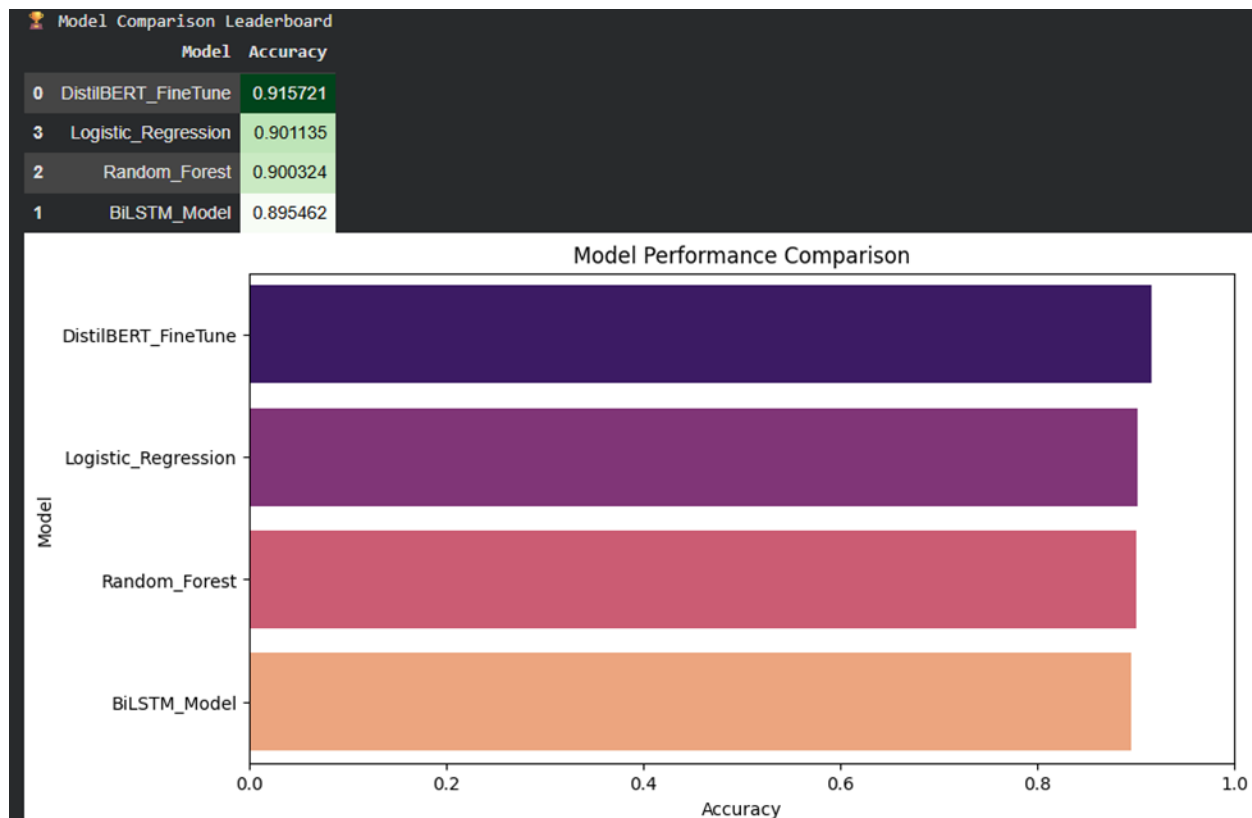


Figure 10 - Model Performance Comparison

Model Performance Comparison

A single performance, Figure 10, comparison leaderboard was created to evaluate and rank all trained models based on accuracy. The models compared include Logistic Regression, Random Forest, BiLSTM, and a fine-tuned DistilBERT transformer model.

The leaderboard results indicate that **DistilBERT Fine-Tuned** achieved the highest accuracy of **91.57%**, outperforming traditional machine learning models and recurrent neural networks.

Logistic Regression and Random Forest demonstrated competitive but slightly lower performance, while the BiLSTM model achieved the lowest accuracy among the evaluated approaches.

This comparison highlights the superiority of transformer-based models in capturing contextual and semantic information in text. While classical models rely on surface-level features, DistilBERT leverages bidirectional contextual embeddings, enabling more accurate detection of nuanced toxic language. Based on these results, DistilBERT was selected as the **champion model** using the MLflow model selection framework.

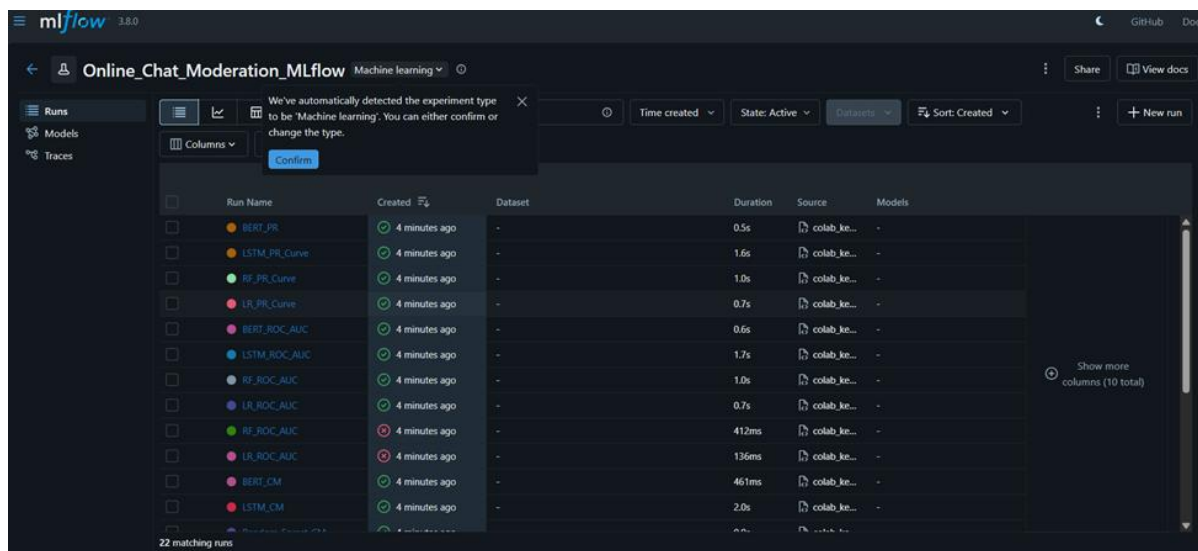


Figure 11 – MLFlow

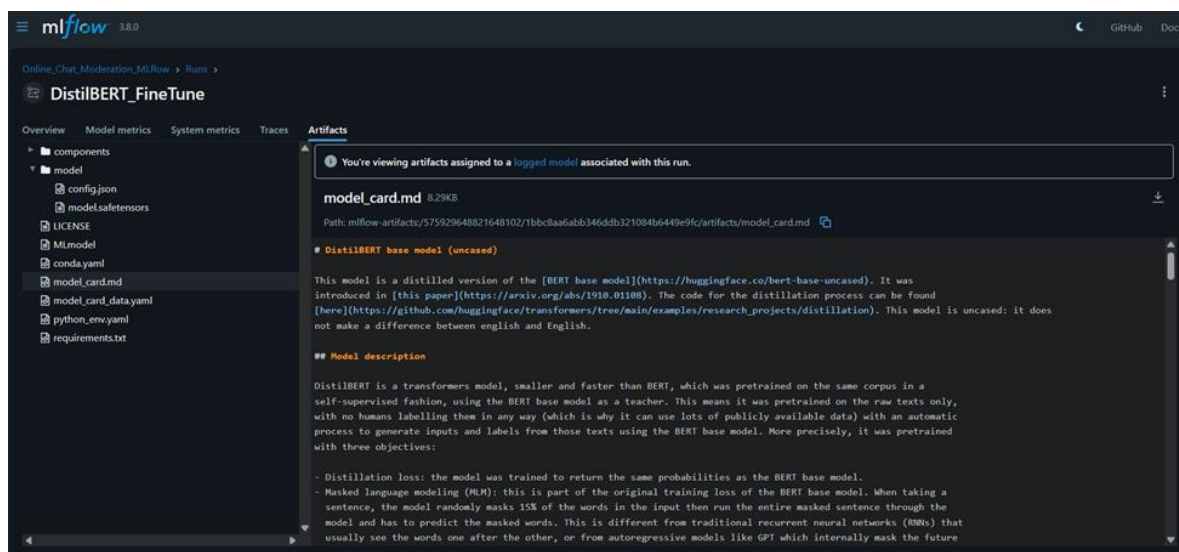


Figure 12 – DistilBERT MLFlow

Experiment Tracking and Model Selection using MLflow

In Figure 11 and 12, All training experiments were systematically logged using **MLflow**, including accuracy, ROC-AUC, precision-recall metrics, and model artifacts. The MLflow dashboard clearly shows multiple experimental runs corresponding to different models and evaluation curves.

MLflow enabled transparent comparison across experiments and ensured reproducibility by linking each run to its training configuration and artifacts. The DistilBERT model consistently demonstrated superior performance across multiple metrics and was therefore promoted to the production stage. This experiment-driven selection process validates the importance of MLflow in supporting objective and auditable model governance within the MLOps pipeline.

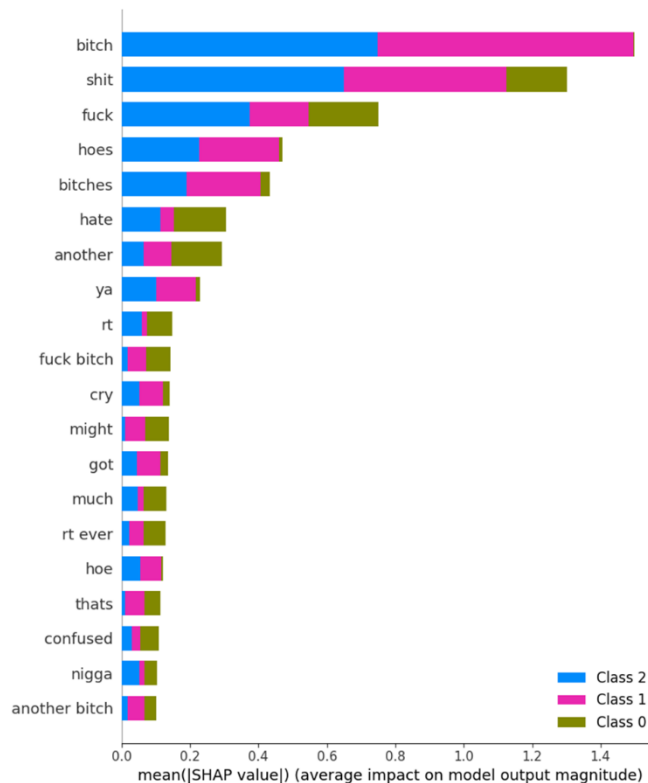


Figure 13 – SHAP Analysis

Explainability Analysis using SHAP

In Figure 13, SHAP-based feature importance analysis was conducted to interpret the predictions of the trained model. The SHAP visualization highlights the most influential words contributing to toxicity predictions across different classes.

Highly offensive and abusive terms exhibit strong positive SHAP values, confirming that the model learns meaningful and intuitive linguistic patterns rather than spurious correlations. The presence of different contribution magnitudes across classes demonstrates the model's ability to distinguish between varying levels of toxicity.

This explainability analysis is critical for ethical AI deployment, as it provides transparency into why specific comments are flagged. By integrating SHAP into the evaluation pipeline, the system ensures that moderation decisions can be justified and audited, aligning with responsible AI principles.

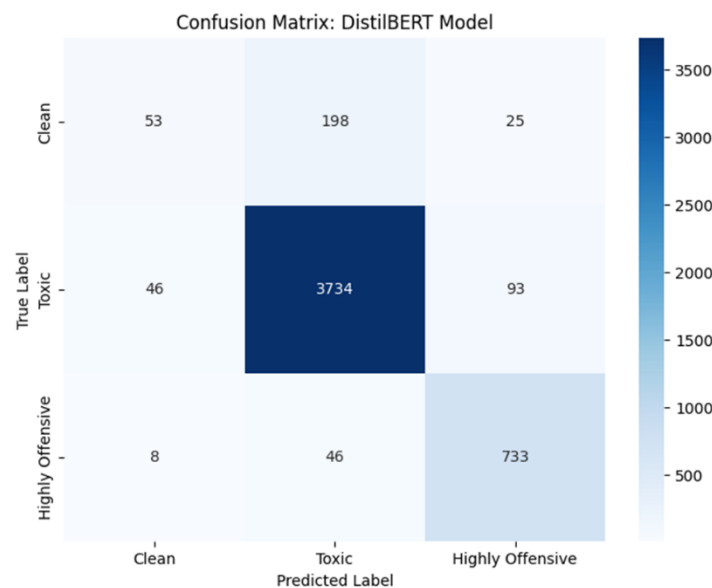


Figure 14 – Confusion Matrix for BERT

Confusion Matrix Analysis of DistilBERT

The confusion matrix in Figure 14 of the DistilBERT model provides deeper insight into class-wise prediction behavior across three categories: **Clean**, **Toxic**, and **Highly Offensive**.

The model shows strong performance in correctly identifying **Toxic** and **Highly Offensive** comments, with a high number of true positives in these categories. A limited number of misclassifications occur primarily between adjacent severity classes, which is expected due to semantic overlap between toxic and highly offensive language.

Importantly, the relatively low misclassification of clean comments as offensive indicates a reduced false-positive rate, which is essential for real-world moderation systems to avoid over-censorship.

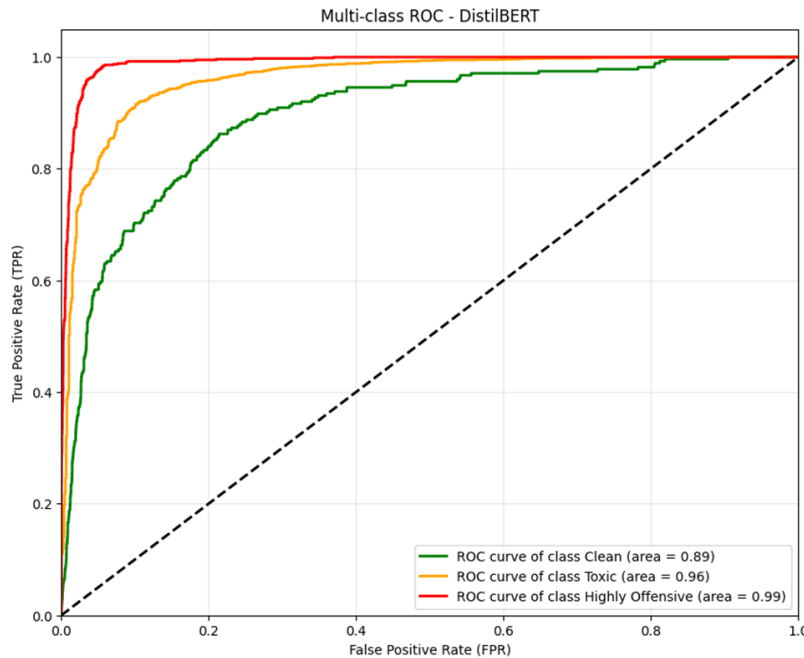


Figure 15 – ROC Curve

ROC Curve Analysis

The multi-class ROC analysis in Figure 15 further confirms the robustness of the DistilBERT model. The area under the ROC curve (AUC) is observed to be:

- **0.89 for Clean**
- **0.96 for Toxic**
- **0.99 for Highly Offensive**

These high AUC values demonstrate excellent class separability, particularly for harmful content categories. The strong ROC performance indicates that the model can effectively distinguish toxic and highly offensive comments from non-toxic ones across different decision thresholds.

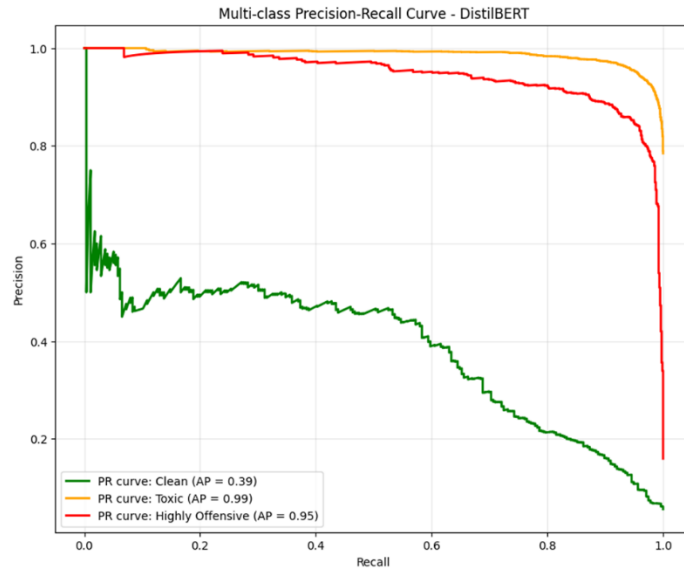


Figure 16 – Precision – Recall Curve

Precision-Recall Curve Analysis

Precision-Recall (PR) curve in Figure 16 provides a more informative evaluation for imbalanced datasets. The DistilBERT model achieves:

- High average precision for **Toxic** and **Highly Offensive** classes
- Lower average precision for the **Clean** class, reflecting the dataset imbalance and the model's focus on detecting harmful content

This behavior is desirable in moderation systems, where prioritizing the detection of toxic content is more critical than perfectly classifying clean comments. The PR curves confirm that the model maintains high precision even at higher recall levels for harmful classes.



CHALLENGES FACED

Developing and deploying a real-world chat moderation system presents several technical and operational challenges, particularly at the intersection of machine learning and MLOps workflows.

Class Imbalance in Toxic Datasets:

A major challenge was the severe class imbalance in toxic comment datasets, where non-toxic samples dominate. This skew can bias models toward the majority class, resulting in poor detection of harmful content. To mitigate this, evaluation metrics beyond accuracy such as precision, recall, and F1-score were consistently tracked using MLflow. Different dataset balancing strategies were versioned using DVC to enable reproducible and transparent experiment comparison.

Evolving Language and Sarcasm:

Online language evolves rapidly through new slang, abbreviations, emojis, and sarcasm, causing trained models to degrade over time. To address this, continuous monitoring was implemented using Evidently AI to detect data and prediction drift. Drift signals were used to trigger retraining, ensuring sustained moderation performance in dynamic environments.

Computational Cost of Transformer Models:

Although transformer-based models such as BERT provided strong accuracy, they introduced high computational costs during training and inference. Efficient experiment tracking and model selection through MLflow helped limit resource usage by promoting only high-performing models to deployment. Docker ensured environment consistency, though inference latency remained a key consideration for real-time use cases.

Explainability in Deep Learning:

Integrating explainability using SHAP with deep models added computational complexity but was essential for transparency and governance. Optimized sampling techniques were applied to reduce overhead, enabling interpretable predictions that support auditability and responsible AI practices.



End-to-End Pipeline Complexity:

Coordinating data versioning, training, deployment, monitoring, and CI/CD across tools such as DVC, MLflow, FastAPI, Docker, Evidently AI, and GitHub Actions introduced architectural complexity. This reinforced the need for standardized pipelines, automated workflows, and strict version control to maintain system consistency and reliability.



CONCLUSION

This project presents a comprehensive, ethical, and scalable AI-based chat moderation system that effectively combines advanced natural language processing models with modern MLOps practices. By leveraging tools such as **DVC for data versioning, Spark for scalable processing, MLflow for experiment tracking and model management, SHAP for explainability, Evidently AI for drift monitoring, and GitHub Actions with Docker for automated deployment**, the system addresses the full lifecycle of machine learning in production.

The integration of explainability and continuous monitoring ensures that the system not only achieves high predictive performance but also aligns with responsible AI principles. The proposed architecture demonstrates how MLOps enables reliable, transparent, and continuously improving moderation systems capable of handling real-world challenges such as evolving language patterns and deployment complexity. Overall, this work highlights the critical role of MLOps in transforming machine learning models into trustworthy and production-ready AI systems for content moderation.

REFERENCES

- [1] P. Fortuna and S. Nunes, "A Survey on Automatic Detection of Hate Speech in Text," *ACM Computing Surveys*, vol. 55, no. 2, pp. 1–37, 2022.
- [2] T. Davidson, D. Warmley, M. Macy, and I. Weber, "Automated Hate Speech Detection and the Problem of Offensive Language," *Proceedings of ICWSM*, pp. 512–515, 2022.
- [3] A. Schmidt and M. Wiegand, "A Survey on Hate Speech Detection Using Natural Language Processing," *Social Network Analysis and Mining*, vol. 13, no. 1, pp. 1–23, 2023.
- [4] Z. Zhao, J. Gao, and X. Huang, "Mitigating Identity Term Bias in Toxic Comment Classification," *Information Processing & Management*, vol. 59, no. 6, 2022.
- [5] S. Risch, A. R. Arora, and R. Krestel, "Transformer-Based Approaches for Abusive Language Detection," *Proceedings of the ACL*, pp. 789–799, 2023.
- [6] A. Vidgen, D. Bontcheva, and L. K. Derczynski, "Challenges and Frontiers in Abusive Content Detection," *Computational Linguistics*, vol. 49, no. 3, pp. 547–580, 2023.
- [7] Y. Cao, L. Wang, and M. Li, "Beyond Classification: Understanding Real-World Toxic Content Moderation Workflows," *Proceedings of the ACL*, pp. 1021–1033, 2024.
- [8] Z. Hu, J. Zhao, and K. Liu, "Toxicity Detection for Free: Leveraging Aligned Large Language Models," *arXiv preprint arXiv:2403.01912*, 2024.
- [9] J. Chan, E. Wallace, and D. Klein, "Detecting Disguised Toxicity in Online Content," *Proceedings of EMNLP*, pp. 456–468, 2024.
- [10] S. M. Lundberg and S. Lee, "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 4765–4774, 2022.
- [11] M. Zangl, P. Holzinger, and H. Müller, "Explainable AI for Toxic Content Moderation in Production Systems," *Springer AI Review*, vol. 18, no. 1, pp. 1–19, 2025.
- [12] R. Sculley, G. Holt, D. Golovin, et al., "Machine Learning Technical Debt Revisited," *Proceedings of the IEEE International Conference on Big Data*, pp. 1–10, 2023.